Кейс: Агентский ИИ-пайплайн полного цикла разработки корпоративных приложений

Для кого: для студентов, желающих попробовать свои силы в решении ИИ-кейса, при работе над которым вы создадите пайплайн для прототипирования приложений.

Для каких направлений: системные аналитики, продакт/проджект-менеджеры, аналитики данных, разработчики ПО (фуллстек), ML-инженеры.

1. Контекст и проблематика

Команда Russtech регулярно получает запросы на создание внутренних цифровых сервисов: различных мобильных приложений, личных кабинетов, административных панелей, систем учета, мониторинга и автоматизации операционных процессов. Даже при наличии формализованных бизнес-требований разработка первого работающего решения требует участия системных аналитиков, дизайнеров, frontend- и backend-разработчиков, специалистов по базам данных, тестированию, информационной безопасности и DevOps.

Чтобы ускорить этот процесс, мы хотим внедрить в работу агентский ИИ-пайплайн, который будет получать на входе формализованные бизнес-требования, а затем преобразовывать их в работающий, протестированный, документированный и развернутый прототип корпоративного приложения.

2. Цель

Разработать агентский процесс, в котором бизнес-требования будут трансформироваться в системный анализ, программный код, инфраструктуру, тесты, документацию и запущенный продукт.

3. Задача

Создать пригодный для практического применения ИИ-пайплайн разработки, который позволит ИИ-инженеру или другому бизнес-пользователю:

1) передать системе формализованные бизнес-требования;
2) получить системный анализ задачи;
3) сформировать архитектуру и структуру решения;
4) создать интерфейсы и программные компоненты;
5) собрать приложение;
6) выполнить автоматизированное тестирование;

7) проверить зависимости и код на известные уязвимости;
8) развернуть результат в тестовой среде;
9) получить продуктовую и пользовательскую документацию;
10) продемонстрировать работающий прототип конечным пользователям.

Бизнес-требования являются единственным предметным входом. При этом допускаются заранее подготовленные технологические шаблоны, правила работы со стеком и корпоративные регламенты, но не являются обязательными.

Созданное решение должно позволять продолжить разработку обычной продуктовой командой. В отдельных случаях полученный прототип должен иметь достаточное качество для ограниченного промышленного использования без полной переработки проекта.

Использование искусственного интеллекта, LLM, субагентов, MCP-серверов, навыков, специализированных моделей и средств генерации кода при решении данного кейса приветствуется и является основной частью задания.

Предмет разработки

Участники создают не отдельное приложение для одного заранее известного сценария, а универсальный агентский конвейер.

На вход пайплайна поступают исключительно бизнес-требования. Способ взаимодействия с системой команда определяет самостоятельно. Это может быть:

- веб-интерфейс;
- агентская среда;
- консольный инструмент;
- расширение среды разработки;
- набор MCP-серверов;
- система навыков и субагентов;
- интеграция с локальным GitLab;
- комбинация нескольких инструментов.

Пайплайн должен самостоятельно или при минимальном участии пользователя пройти последовательность этапов разработки и сохранить полученные артефакты в репозитории.

Система не должна превращать работу с бизнес-требованиями в анкету из сотен обязательных вопросов. Допускаются уточняющие вопросы, если они существенно влияют на архитектуру, безопасность или пользовательский сценарий. Их количество, обоснованность и влияние на итоговый результат также могут учитываться при оценке.

Основной сценарий работы

Этап 1. Прием и анализ требований

Система принимает формализованное описание бизнес-задачи и определяет:

- цели продукта;
- роли пользователей;
- функциональные и нефункциональные требования;
- ограничения;
- основные сущности;
- пользовательские сценарии;
- интеграционные точки;
- требования к безопасности;
- критерии приемки;
- риски и неоднозначности.

Этап 2. Системный анализ

По результатам обработки требований должен быть сформирован отдельный документ системного анализа в формате Markdown. Документ должен соответствовать переданным организаторами правилам и включать:

- назначение системы;
- границы решения;
- роли и права доступа;
- пользовательские сценарии;
- функциональные требования;
- нефункциональные требования;
- описание компонентов;
- модель данных;
- API-контракты;
- события и интеграции;
- ограничения и проверки входных данных;
- обработку ошибок;
- критерии приемки;
- допущения и открытые вопросы.

Каждому функциональному требованию необходимо присваивать идентификатор. Допущения системы должны быть явно отделены от требований заказчика, а критические неоднозначности — вынесены на уточнение.

Документация, API-контракты, модель данных и код должны описывать одно и то же поведение. Неприменимые разделы шаблонов допустимо отмечать

с обоснованием, без формального заполнения и выдумывания отсутствующих компонентов.

Этап 3. Проектирование интерфейса

Пайплайн должен сформировать пользовательские потоки, экраны, состояния и компоненты интерфейса. Для web-интерфейса обязательно использование UI Kit, который команда может выбрать самостоятельно. Пайплайн должен быть настроен на правила этой выбранной дизайн-системы и корректно применять ее компоненты.

Допускается создание отдельных макетов интерфейса с сохранением исходников в репозитории. Сгенерированные экраны должны учитывать:

- основные и альтернативные сценарии;
- пустые состояния;
- загрузку данных;
- ошибки;
- ограничения доступа;
- адаптивность;
- единообразие компонентов;
- понятную обратную связь для пользователя.

Для интерфейсов важно проверять фактическое выполнение пользовательского сценария: действия должны обращаться к работающему backend, данные — сохраняться, ошибки и ограничения доступа — обрабатываться. Заглушки допустимы для обозначенных внешних интеграций, но не вместо основной бизнес-логики приложения.

Этап 4. Разработка компонентов

В репозитории должны появляться отдельные компоненты решения:

- системная и продуктовая документация;
- frontend;
- backend;
- база данных и миграции;
- мобильное приложение, если оно предусмотрено требованиями;
- тесты;
- инфраструктурные файлы;
- конфигурация CI/CD;
- mock-сервисы для недоступных интеграций.

Компоненты должны собираться в единый работающий проект.

Этап 5. Тестирование и безопасность

Пайплайн должен выполнять доступные автоматизированные проверки:

- сборка проекта;
- модульные тесты;
- интеграционные тесты;
- проверка API;
- проверка основных пользовательских сценариев;
- статический анализ кода;
- проверка зависимостей;
- поиск известных уязвимостей и CVE;
- формирование отчета о результатах проверок.

Команда самостоятельно выбирает инструменты тестирования и безопасности. Важно, чтобы результаты проверок были воспроизводимыми и сохранялись в GitLab.

Проверки должны подтверждать выполнение исходных требований, а не только успешную сборку проекта. Отчет должен содержать результаты тестов, обнаруженные дефекты и уязвимости, их критичность и итоговый статус проверки. Наличие подключенного сканера само по себе не является подтверждением успешного прохождения тестов.

Этап 6. Сборка и развертывание

Сгенерированный проект должен быть контейнеризирован с использованием Docker. Минимально ожидаются:

- Dockerfile для необходимых компонентов;
- docker-compose.yml для локального запуска;
- конфигурация переменных окружения;
- миграции и инициализация базы данных;
- health checks;
- инструкция по запуску;
- тестовые данные.

На площадке хакатона будет использоваться локальный GitLab. Командам необходимо настроить GitLab CI/CD либо предоставить сопоставимый автоматизированный механизм сборки, тестирования и развертывания. Итоговый прототип должен запускаться и предоставляться эксперту.

Запуск экспертом осуществляется из чистого окружения по инструкции из репозитория. Должны выполняться сборка, миграции, загрузка тестовых данных и проверки работоспособности без обязательного ручного исправления исходного кода. Секреты и реальные учетные данные не должны храниться в репозитории.

Этап 7. Документация

В результате работы должны быть сформированы:

- документ системного анализа;
- продуктовое описание;
- описание архитектуры;
- описание API;
- модель данных;
- инструкция по запуску;
- руководство пользователя;
- руководство по дальнейшей разработке;
- отчет о тестировании;
- отчет о проверке безопасности;
- перечень известных ограничений.

Технологический стек

Целевой стек решения	
Angular	Frontend 20.1.6
C# / ASP.NET Core	Backend .NET 10, C# 14
PostgreSQL / PostGIS	Данные и геоинформационные операции PostgreSQL 17.9 (Debian 17.9-1.pgdg12+1) postgis-jdbc 2.5.0
Kotlin	Мобильное Android-приложение 2.2.21
Kafka	Обмен событиями между компонентами, если он предусмотрен архитектурой
Docker / Docker Compose	Контейнеризация и локальный запуск
GitLab CI/CD	Сборка, тестирование, проверки и развертывание 19.4.2
LLM и агентские инструменты	Анализ требований и генерация артефактов Qwen3.8-27B

Требования к API и событиям

Создаваемое решение должно иметь документированный API. В зависимости от выбранной архитектуры необходимо предусмотреть:

- OpenAPI-спецификацию;
- модели запросов и ответов;
- обязательные и необязательные поля;
- ограничения длины и форматов;
- регулярные выражения для проверяемых строковых значений;
- единый формат ошибок;
- аутентификацию и авторизацию;
- ограничение количества запросов;
- идемпотентность критичных операций;
- журналирование;
- версионирование API.

При использовании Kafka события должны быть небольшими и предметными, иметь документированную схему, версию, идентификатор, время возникновения и правила повторной обработки.

Реальная интеграция с внешними промышленными системами в рамках хакатона не требуется. Такие системы должны заменяться документированными mock-сервисами или адаптерами.

4. Формат участия

При решении кейса команда может выбрать один из двух подходов.

1. Полный пайплайн

Реализовать всю последовательность от бизнес-требований до работающего и развернутого приложения.

2. Специализированный участок

Сфокусироваться на одном или нескольких этапах и реализовать их с высоким качеством, например:

- анализ бизнес-требований;
- системный анализ;
- проектирование интерфейсов;
- генерация frontend;
- генерация backend;
- проектирование базы данных;

· тестирование;
· анализ безопасности;
· GitLab CI/CD;
· документирование.

Отсутствующие этапы не дисквалифицируют команду, но баллы за них не начисляются. Победить сможет как команда с наиболее полным решением, так и команда, создавшая выдающийся специализированный компонент.

5. Ожидаемый результат

Команда должна представить:

· работающий агентский пайплайн или его значимую часть;
· репозиторий в локальном GitLab;
· сформированные документы;
· исходный код компонентов;
· настроенные тесты и проверки;
· Docker-окружение;
· CI/CD-конфигурацию;
· работающий прототип;
· пользовательскую документацию;
· отчет о расходе токенов, времени и количестве попыток;
· демонстрацию прохождения проверочного кейса (см. **Приложение № 1** к этому документу).

Команда должна показать не только заранее подготовленный результат, но и работу пайплайна на выданных бизнес-требованиях.

Решения команд будут проходить проверку на двух проверочных кейсах:
• кейс № 1 — предоставляется командам для предварительной подготовки;
• кейс № 2 — остается нераскрытым контрольным заданием до начала итоговой проверки. Запуск и сборка проекта по этому кейсу выполняются строго в момент проведения оценки решений.

6. Критерии оценки (максимальная оценка — 100 баллов)

1. Качество анализа — 20 баллов

· корректность интерпретации требований;
· полнота системного анализа;

- качество критериев приемки;
- выявление неоднозначностей;
- соответствие предоставленному регламенту.

2. Качество продукта и интерфейса — 15 баллов

- удобство пользовательского сценария;
- строгое соответствие выбранному UI Kit;
- полнота состояний интерфейса;
- качество мобильного и web-прототипов;
- соответствие интерфейса исходным требованиям.

3. Архитектура и разработка — 20 баллов

- качество структуры решения;
- корректность API и модели данных;
- разделение компонентов;
- качество сгенерированного кода;
- возможность дальнейшего сопровождения.

4. Тестирование и безопасность — 15 баллов

- автоматизированные тесты;
- проверка основных сценариев;
- статический анализ;
- проверка зависимостей и известных CVE;
- понятность сформированных отчетов.

5. Инфраструктура — 10 баллов

- Docker-контейнеризация;
- воспроизводимость запуска;
- GitLab CI/CD;
- автоматическая сборка и развертывание;
- сохранение артефактов и результатов проверок.

6. Документация — 10 баллов

- продуктовое описание;
- техническая документация;
- OpenAPI;
- инструкция по запуску;
- пользовательское руководство.

7. Эффективность пайплайна — 10 баллов

- время получения результата;
- расход токенов;
- количество запусков и попыток;
- объем ручного вмешательства;
- стабильность повторного запуска.

Порядок замера эффективности

Для разработки и отладки участникам предоставляется доступ к модели без конкурсного учета расхода. Во время итоговой проверки используется отдельный демонстрационный ключ. Через него организаторы фиксируют:

- количество входных и выходных токенов;
- число обращений к модели;
- число повторных попыток;
- длительность генерации;
- длительность полного прохождения пайплайна;
- итоговый статус сборки, тестирования и развертывания.

Результаты замеров могут отображаться на общем экране после прохождения проверочного кейса. Большое число попыток и перебор решений не должны давать преимущество перед более точным и экономичным пайплайном.

Приложение № 1

Проверочный кейс: Помощник управления платными парковками Санкт-Петербурга

1. Контекст и проблематика

Сотрудники наших технических служб ежедневно выполняют работы на нескольких объектах Санкт-Петербурга, используя служебные автомобили. В течение рабочего дня водитель может перемещаться между объектами, расположенными в пределах одной или нескольких зон платной парковки.

Сейчас водитель самостоятельно контролирует необходимость оплаты, время окончания оплаченного периода и переход между парковочными зонами. Это увеличивает вероятность повторной оплаты, пропуска окончания сессии и получения штрафа.

2. Цель проверочного кейса

Создать прототип мобильного помощника, который определяет нахождение автомобиля в зоне платной парковки, отслеживает активную парковочную сессию и помогает водителю понять, требуется ли новая оплата.

Этот кейс используется для проверки способности пайплайна преобразовать бизнес-требования в системный анализ, интерфейс, программные компоненты, тесты, документацию и работающий прототип.

3. Функциональные требования

Система должна:

- определять местоположение автомобиля;
- определять нахождение автомобиля в платной парковочной зоне;
- показывать номер и границы зоны;
- отображать доступные параметры парковки;
- хранить информацию об активной парковочной сессии;
- отслеживать перемещение автомобиля;
- определять выход из текущей зоны;
- определять переход в другую парковочную зону;
- контролировать время окончания оплаченной сессии;
- предупреждать о необходимости проверки или новой оплаты;
- предоставлять переход к официальному сервису оплаты;
- отображать маршрут и историю парковочных сессий.

Примеры уведомлений:

· «Вы въехали в зону платной парковки».
· «Парковка оплачена до 15:20».
· «До окончания парковки осталось 10 минут».
· «Вы покинули текущую парковочную зону».
· «Вы въехали в другую парковочную зону. Проверьте необходимость новой оплаты».

4. Ожидаемые компоненты

Для проверочного кейса должны быть сформированы:

· мобильное приложение на Kotlin;
· административный web-интерфейс на Angular;
· backend на C# / ASP.NET Core;
· PostgreSQL/PostGIS;
· документированный API;
· модель событий;
· Docker-окружение;
· тестовые данные;
· автоматизированные тесты;
· CI/CD-пайплайн;
· системная, продуктовая и пользовательская документация.

Web-интерфейс должен позволять просматривать автомобили, парковочные зоны, активные сессии, маршруты, события и историю. Мобильный интерфейс должен содержать карту, текущую зону, статус сессии, оставшееся время, уведомления и действие для перехода к оплате.

5. Источники данных и интеграции

В проверочном сценарии могут использоваться:

- открытые данные о парковочных зонах Санкт-Петербурга;
- GPS-координаты или демонстрационный mock-маршрут;
- геоданные парковочных зон;
- официальный сервис парковок Санкт-Петербурга;
- приложение «Парковки Санкт-Петербурга» или «Парковки России»;
- сервис и API «Паркоматика».

«Паркоматика» может рассматриваться как возможная интеграционная основа для работы с организациями, автомобилями, водителями, зонами и парковочными сессиями.

Получение промышленного доступа и выполнение реальной оплаты не требуются. Интеграция должна быть реализована через адаптер и mock-сервис,

чтобы внешнего поставщика можно было заменить без изменения основной бизнес-логики. Собственный платежный механизм разрабатывать не требуется.

6. Демонстрационный сценарий

Во время демонстрации система должна обработать последовательность событий:

1) служебный автомобиль движется по заданному маршруту;
2) автомобиль въезжает в зону платной парковки;
3) система определяет зону и уведомляет водителя;
4) водитель запускает или подтверждает парковочную сессию;
5) система показывает время ее окончания;
6) за 10 минут до окончания поступает предупреждение;
7) автомобиль покидает зону или перемещается в другую зону;
8) система определяет изменение и предлагает проверить необходимость новой оплаты;
9) события отображаются в административном интерфейсе и сохраняются в истории.